

A Distributed Real-Time Intrusion Detection System on a Jetson Orin Nano Super Edge Cluster using Kafka and PySpark

Thai Nguyen Vu¹ Dung Van Bui² Nhut Tri Do² Van Du Nguyen³
SOICT 2026

¹Campus in Ho Chi Minh City, University of Transport and Communications, Ho Chi Minh City, Vietnam

²University of Information Technology, VNU-HCM, Ho Chi Minh City, Vietnam

³Faculty of Information Technology, Nong Lam University, Ho Chi Minh City, Vietnam

2026-09-29

A Distributed Real-Time Intrusion Detection System on a Jetson Orin Nano Super Edge Cluster using Kafka and PySpark

A Distributed Real-Time Intrusion Detection System on a Jetson Orin Nano Super Edge Cluster using Kafka and PySpark

Thai Nguyen Vu¹ Dung Van Bui² Nhut Tri Do² Van Du Nguyen³
SOICT 2026

¹Campus in Ho Chi Minh City, University of Transport and Communications, Ho Chi Minh City, Vietnam

²University of Information Technology, VNU-HCM, Ho Chi Minh City, Vietnam

³Faculty of Information Technology, Nong Lam University, Ho Chi Minh City, Vietnam

Good morning. I will show what a second edge board actually buys you when you split an ML intrusion detector across two Jetson Orin Nano Super devices. Seven minutes, four results.

I. PROBLEM AND CONTRIBUTIONS

2026-09-29

A Distributed Real-Time Intrusion Detection System on a Jetson
Orin Nano Super Edge Cluster using Kafka and PySpark
└ I. PROBLEM AND CONTRIBUTIONS

I. PROBLEM AND
CONTRIBUTIONS

Edge intrusion detection: three open gaps

Detectors now *fit* on single-board hardware — but a monolithic one classifies **every** flow and saturates an 8 GB ARM board.

What the literature does not tell us

- 1. Everything is measured on a **single device**.
- 2. Throughput, latency and **energy** are never reported together.
- 3. Offline scores come from **leaky protocols**.

Our question

How should inference be *placed* across a constrained two-node edge cluster?

Our answer

Split it: a cheap **autoencoder gate** on board #1, the heavy **PySpark classifier** on board #2, joined by a Kafka topic.

2026-09-29

A Distributed Real-Time Intrusion Detection System on a Jetson Orin Nano Super Edge Cluster using Kafka and PySpark

I. PROBLEM AND CONTRIBUTIONS

└ Edge intrusion detection: three open gaps

Three gaps: single-device evaluation, energy never reported next to latency, and offline numbers taken from leaky pipelines. Which model to deploy is the other half of the question. We attack all three with one system.

Edge intrusion detection: three open gaps

Detectors now *fit* on single-board hardware — but a monolithic one classifies **every** flow and saturates an 8 GB ARM board.

What the literature does not tell us

- 1. Everything is measured on a **single device**.
- 2. Throughput, latency and **energy** are never reported together.
- 3. Offline scores come from **leaky protocols**.

Our question

How should inference be placed across a constrained two-node edge cluster?

Our answer

Split it: a cheap **autoencoder gate** on board #1, the heavy **PySpark classifier** on board #2 joined by a Kafka topic.

1. A **two-stage distributed IDS**: anomaly gate + Spark classifier, physically separated across two boards.
2. **Three deployment modes** for a two-node Jetson cluster — pipeline split, horizontal Kafka scaling, Spark standalone cluster.
3. An **end-to-end evaluation**: throughput, latency percentiles, per-node CPU/RAM *and* energy per verdict.
4. **Leakage-aware provenance** — the deployed model is exactly the one our companion study selected under a leakage-free protocol.

2026-09-29

A Distributed Real-Time Intrusion Detection System on a Jetson Orin Nano Super Edge Cluster using Kafka and PySpark

I. PROBLEM AND CONTRIBUTIONS

Contributions

Four contributions. The fourth matters: the model we deploy is not a fresh model, it is the one that survived a strict offline audit.

1. A **two-stage distributed IDS**: anomaly gate + Spark classifier, physically separated across two boards.
2. **Three deployment modes** for a two-node Jetson cluster — pipeline split, horizontal Kafka scaling, Spark standalone cluster.
3. An **end-to-end evaluation**: throughput, latency percentiles, per-node CPU/RAM *and* energy per verdict.
4. **Leakage-aware provenance** — the deployed model is exactly the one our companion study selected under a leakage-free protocol.

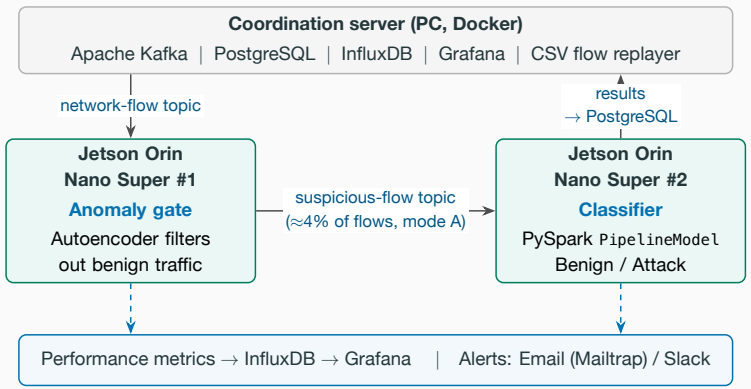
2026-09-29

A Distributed Real-Time Intrusion Detection System on a Jetson Orin Nano Super Edge Cluster using Kafka and PySpark
└─ II. SYSTEM AND EVALUATION SETUP

II. SYSTEM AND EVALUATION
SETUP

II. SYSTEM AND EVALUATION SETUP

Split-deployment architecture



The gate's MSE threshold is calibrated *offline* on a held-out benign validation split — committed before evaluation, never tuned on the test set.

CICIDS2017: Sharafaldin et al., ICISSP 2018 · Kafka: Kreps et al., NetDB 2011 · Spark: Zaharia et al., HotCloud 2010.

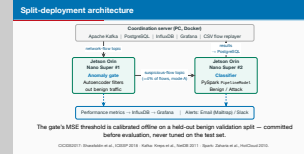
2026-09-29

A Distributed Real-Time Intrusion Detection System on a Jetson Orin Nano Super Edge Cluster using Kafka and PySpark

II. SYSTEM AND EVALUATION SETUP

Split-deployment architecture

Host runs the infrastructure. Board one gates, board two classifies. Only suspicious flows cross the Kafka topic between them. Everything is instrumented into Grafana.



Three deployment modes, one workload

Single node full pipeline in one process — the reference.

A: Pipeline split gate on #1, classifier on #2 (our design).

B: Horizontal both boards run the *full* role, sharing a Kafka consumer group.

C: Spark cluster host as master, both boards as workers.

Throughput is counted as **final verdicts per second** — a flow is decided at the gate or at the classifier — so forwarded flows are never double-counted.

Setup

2× Jetson Orin Nano Super
(6-core A78AE, 8 GB, ≈67 TOPS)
PySpark 3.4.1, Kafka 3.5, Wi-Fi LAN
CICIDS2017 replay at 100 flows/s
≥3 repetitions, warmup excluded
Energy via tegrastats, idle subtracted

2026-09-29

A Distributed Real-Time Intrusion Detection System on a Jetson Orin Nano Super Edge Cluster using Kafka and PySpark

II. SYSTEM AND EVALUATION SETUP

Three deployment modes, one workload

Four configurations, identical workload. The verdict-per-second definition is what makes the modes comparable.

Three deployment modes, one workload

Single node full pipeline in one process — the reference.

A: Pipeline split gate on #1, classifier on #2 (our design).

B: Horizontal both boards run the full role, sharing a Kafka consumer group.

C: Spark cluster host as master, both boards as workers.

Throughput is counted as **final verdicts per second** — a flow is decided at the gate or at the classifier — so forwarded flows are never double-counted.

Setup

2× Jetson Orin Nano Super
(6-core A78AE, 8 GB, ≈67 TOPS)
PySpark 3.4.1, Kafka 3.5, Wi-Fi LAN
CICIDS2017 replay at 100 flows/s
≥3 repetitions, warmup excluded
Energy via tegrastats, idle subtracted

2026-09-29

A Distributed Real-Time Intrusion Detection System on a Jetson Orin Nano Super Edge Cluster using Kafka and PySpark

└ III. RESULTS AND CONCLUSION

III. RESULTS AND CONCLUSION

III. RESULTS AND CONCLUSION

Result 1: splitting the pipeline buys 24× throughput

Mode	Verdicts/s	p95 (ms)	J / verdict
Single node	2.6	12.3	2.49
A: Split	62.5	13.1	0.18
B: Horizontal	5.9	21.6	2.25
C: Spark cluster	90.0	23.9	0.12

Verdict: the final benign/attack decision for one flow.
p95: 95% of flows get their verdict faster than this.

- Split: 24× throughput, same p95. The gate settles 95.8% of flows before Spark.
- Horizontal: only 2.3× — every board still runs Spark.
- Cluster: faster on average but unstable (±25.6).

Pipeline split is the recommended default: highest throughput with the smallest run-to-run spread.

2026-09-29

A Distributed Real-Time Intrusion Detection System on a Jetson Orin Nano Super Edge Cluster using Kafka and PySpark

III. RESULTS AND CONCLUSION

Result 1: splitting the pipeline buys 24× throughput

This is the core table. Twenty-four times, at the same p95. Mode B proves the gain comes from role separation, not from adding a board. Spreads: single 2.60 plus-minus 0.41, split 62.5 plus-minus 0.6, horizontal 5.9 plus-minus 1.4, cluster 90.0 plus-minus 25.6 flows per second; p95 spreads 3.0, 2.6, 19.2, 17.8 ms. Gate skip is 95.8 percent in split mode and 97.9 in cluster mode; in single and horizontal modes the gate runs inline. Energy is total-board per verdict: the active power delta was below the tegrastats noise floor at this load, so the saving comes from amortizing idle power over 24 to 35 times more verdicts. Cluster mode is for models that exceed single-node memory.

Result 1: splitting the pipeline buys 24× throughput

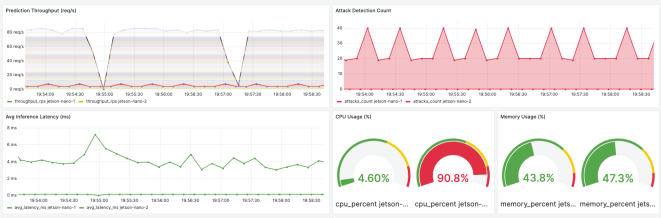
Mode	Verdicts/s	p95 (ms)	J / verdict
Single node	2.6	12.3	2.49
A: Split	62.5	13.1	0.18
B: Horizontal	5.9	21.6	2.25
C: Spark cluster	90.0	23.9	0.12

Verdict: the final benign/attack decision for one flow.
p95: 95% of flows get their verdict faster than this.

Pipeline split is the recommended default: highest throughput with the smallest run-to-run spread.

- Split: 24× throughput, same p95. The gate settles 95.8% of flows before Spark.
- Horizontal: only 2.3× — every board still runs Spark.
- Cluster: faster on average but unstable (±25.6).

Result 2: the load asymmetry is visible live



At peak load, split mode:

- gate node: **4.6% CPU**
- classifier node: **90.8% CPU**

The classifier dominates the compute budget — which is exactly why moving it off the ingest path pays.

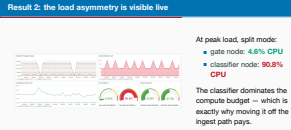
2026-09-29

A Distributed Real-Time Intrusion Detection System on a Jetson Orin Nano Super Edge Cluster using Kafka and PySpark

III. RESULTS AND CONCLUSION

Result 2: the load asymmetry is visible live

Live Grafana at peak load. Under five versus ninety-one percent. The asymmetry is the whole argument for the split. Add: the gate is a tunable filter, not a fixed point — backup slide has the operating curve.



Result 3: what the gate lets through

Gate threshold (benign quantile)	Attacks forwarded	False alarms	Flows skipped
0.99	74.2%	1.01%	88.0%
0.995 (deployed)	72.3%	0.53%	88.7%
0.999	58.1%	0.10%	91.2%

Online, on the CICIDS2017 test split; threshold fixed on benign validation data only.

- A flow the gate skips is final: **about 28% of attack flows** never reach the classifier at the deployed threshold.
- Lowering the threshold forwards more attacks but sends more traffic to Spark.
- The operator picks the point that fits the budget.

The throughput gain is paid for in recall: the gate is a **tunable trade-off**, not a free filter.

2026-09-29

A Distributed Real-Time Intrusion Detection System on a Jetson Orin Nano Super Edge Cluster using Kafka and PySpark

III. RESULTS AND CONCLUSION

Result 3: what the gate lets through

Say this before anyone asks. The numbers come from results/ml_08_anomaly_gate/gate_operating_points.csv; the deployed threshold is jetson/model/anomaly_threshold.json, quantile 0.995. The skip ratio here is offline on the test split; the live replay skipped 95.8% of a different traffic sample, so the two ratios need not match. Full curve on the backup slide.

Result 3: what the gate lets through

Gate threshold (benign quantile)	Attacks forwarded	False alarms	Flows skipped
0.99	74.2%	1.01%	88.0%
0.995 (deployed)	72.3%	0.53%	88.7%
0.999	58.1%	0.10%	91.2%

- A flow the gate skips is final: **about 28% of attack flows** never reach the classifier at the deployed threshold.
- Lowering the threshold forwards more attacks but sends more traffic to Spark.
- The operator picks the point that fits the budget.

The throughput gain is paid for in recall: the gate is a **tunable trade-off**, not a free filter.

Result 4: the price of one unified stack

We serve with Spark for consistency with distributed training. What does that cost? Same forest, same 29 deployed features, same board:

Engine	Flows/s	p95 (ms)	RAM (%)	mJ per flow
PySpark (deployed)	22.6	532	24.3	71.6
scikit-learn	204.4	49.8	13.3	2.91
ONNX Runtime	7870.8	1.1	15.7	0.06

mJ per flow: energy above the board's idle power.

ONNX Runtime sustains 348× the deployed PySpark throughput. We report this as a *target for the next iteration* — export to ONNX/TensorRT for serving; keep Spark for training.

2026-09-29

A Distributed Real-Time Intrusion Detection System on a Jetson Orin Nano Super Edge Cluster using Kafka and PySpark

III. RESULTS AND CONCLUSION

Result 4: the price of one unified stack

The JVM costs us three orders of magnitude, and that sets the roadmap.

Result 4: the price of one unified stack				
We serve with Spark for consistency with distributed training. What does that cost? Same forest, same 29 deployed features, same board:				
Engine	Flows/s	p95 (ms)	RAM (%)	mJ per flow
PySpark (deployed)	22.6	532	24.3	71.6
scikit-learn	204.4	49.8	13.3	2.91
ONNX Runtime	7870.8	1.1	15.7	0.06
mJ per flow: energy above the board's idle power.				
ONNX Runtime sustains 348× the deployed PySpark throughput. We report this as a target for the next iteration — export to ONNX/TensorRT for serving; keep Spark for training.				

Conclusion

What we showed

- A reproducible distributed edge IDS: Kafka + PySpark on a Jetson Orin Nano Super cluster.
- Splitting gate and classifier across boards: **2.6 → 62.5 flows/s** at p95 ≈13 ms, gate skipping 95.8%.
- Energy per verdict: **2.5 J → 0.18 J**.

Limitations

- Two-node lab cluster over Wi-Fi; replayed traffic.
- Gate forwards 72% of attacks at the deployed threshold.
- GPU unused (CPU-only pipeline); evasion untested.

Next

- ONNX/TensorRT serving; live traffic; retraining under drift.

Thank you — questions?

2026-09-29

A Distributed Real-Time Intrusion Detection System on a Jetson Orin Nano Super Edge Cluster using Kafka and PySpark

III. RESULTS AND CONCLUSION

Conclusion

Recap the three numbers: twenty-four times, thirteen milliseconds, ninety-six percent skipped. Then stop talking.

What we showed ■ A reproducible distributed edge IDS: Kafka + PySpark on a Jetson Orin Nano Super cluster. ■ Splitting gate and classifier across boards: 2.6 → 62.5 flows/s at p95 ≈13 ms, gate skipping 95.8%. ■ Energy per verdict: 2.5 J → 0.18 J.	Limitations ■ Two-node lab cluster over Wi-Fi; replayed traffic. ■ Gate forwards 72% of attacks at the deployed threshold. ■ GPU unused (CPU-only pipeline); evasion untested. Next ■ ONNX/TensorRT serving; live traffic; retraining under drift. Thank you — questions?
---	--

Key references

[1] Sharafaldin, I., Lashkari, A. H. & Ghorbani, A. A. *Toward generating a new intrusion detection dataset and intrusion traffic characterization*. ICISSP 2018. **CICIDS2017**

[2] Shi, W. et al. *Edge computing: Vision and challenges*. IEEE Internet of Things Journal 3(5), 2016. **edge setting**

[3] Chandola, V., Banerjee, A. & Kumar, V. *Anomaly detection: A survey*. ACM Computing Surveys 41(3), 2009. **anomaly gate**

[4] Lundberg, S. M. & Lee, S.-I. *A unified approach to interpreting model predictions*. NeurIPS 2017. **SHAP Top-30**

[5] Zaharia, M. et al. *Spark: Cluster computing with working sets*. HotCloud 2010. **Spark**

[6] Kreps, J., Narkhede, N. & Rao, J. *Kafka: a Distributed Messaging System for Log Processing*. NetDB 2011. **Kafka**

[7] Mohan Kiran Kumar, K. et al. *Distributed Intrusion Detection System using Kafka and Spark Streaming*. ICVADV 2025. **closest system**

[8] Arp, D. et al. *Dos and Don'ts of Machine Learning in Computer Security*. USENIX Security 2022. **leakage-aware protocol**

2026-09-29

A Distributed Real-Time Intrusion Detection System on a Jetson Orin Nano Super Edge Cluster using Kafka and PySpark

└─III. RESULTS AND CONCLUSION

└─Key references

Not presented — shown while taking questions, so sources are on screen if the audience asks. Full list is in the paper.

Key references	
[1] Sharafaldin, I., Lashkari, A. H. & Ghorbani, A. A. Toward generating a new intrusion detection dataset and intrusion traffic characterization. ICISSP 2018.	CICIDS2017
[2] Shi, W. et al. Edge computing: Vision and challenges. IEEE Internet of Things Journal 3(5), 2016.	edge setting
[3] Chandola, V., Banerjee, A. & Kumar, V. Anomaly detection: A survey. ACM Computing Surveys 41(3), 2009.	anomaly gate
[4] Lundberg, S. M. & Lee, S.-I. A unified approach to interpreting model predictions. NeurIPS 2017.	SHAP Top-30
[5] Zaharia, M. et al. Spark: Cluster computing with working sets. HotCloud 2010.	Spark
[6] Kreps, J., Narkhede, N. & Rao, J. Kafka: a Distributed Messaging System for Log Processing. NetDB 2011.	Kafka
[7] Mohan Kiran Kumar, K. et al. Distributed Intrusion Detection System using Kafka and Spark Streaming. ICVADV 2025.	closest system
[8] Arp, D. et al. Dos and Don'ts of Machine Learning in Computer Security. USENIX Security 2022.	leakage-aware protocol

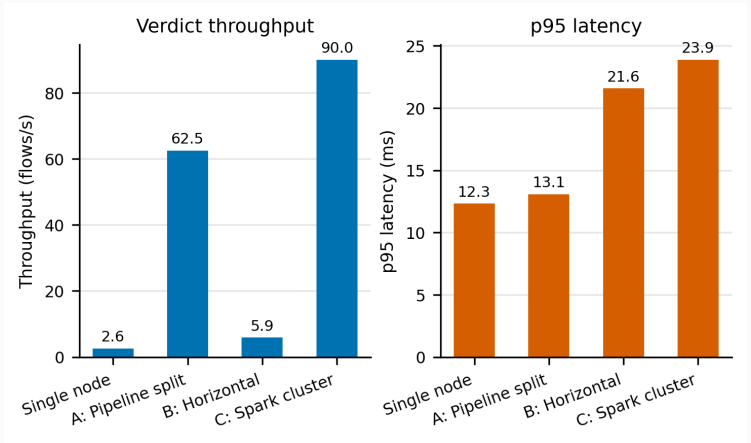
2026-09-29

A Distributed Real-Time Intrusion Detection System on a Jetson Orin Nano Super Edge Cluster using Kafka and PySpark

Backup slides

Backup slides

Backup: deployment-mode comparison chart

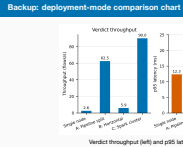


Verdict throughput (left) and p95 latency (right).

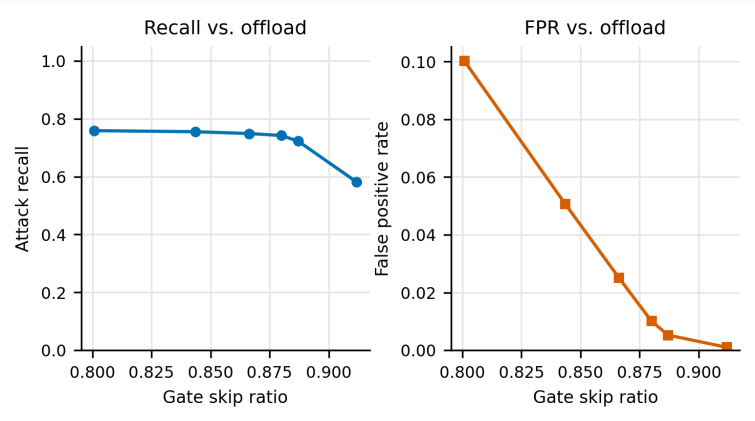
2026-09-29

A Distributed Real-Time Intrusion Detection System on a Jetson Orin Nano Super Edge Cluster using Kafka and PySpark

Backup: deployment-mode comparison chart



Backup: the gate is an operating curve, not a point

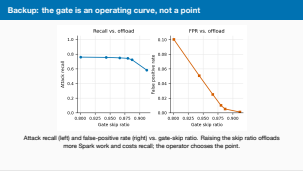


Attack recall (left) and false-positive rate (right) vs. gate-skip ratio. Raising the skip ratio offloads more Spark work and costs recall; the operator chooses the point.

2026-09-29

A Distributed Real-Time Intrusion Detection System on a Jetson Orin Nano Super Edge Cluster using Kafka and PySpark

Backup: the gate is an operating curve, not a point



Backup: which model, and why

Configuration	F1	Features	Size (MB)	Edge rationale
Baseline + XGBoost	0.9971	77	2.32	Reference accuracy
SHAP Top-30 + XGBoost	0.9970	30	0.003 [†]	Explainable; de- ployed feature set
RF Top-30 + XGBoost	0.9922	30	2.52	Embedded ranking
PCA k=40 + XGBoost	0.9939	40	3.82	Unsupervised reduc- tion

We deploy **SHAP Top-30 with Random Forest** (offline F1 0.9955): a native Spark ML estimator, so the exported PipelineModel loads on ARM64 without the native libraries Spark's XGBoost integration needs. The input vector has **29** features — destination_port is in the SHAP ranking but excluded as label-leaking.

[†] Distributed saving under-reports size; true size ≈2.5 MB.

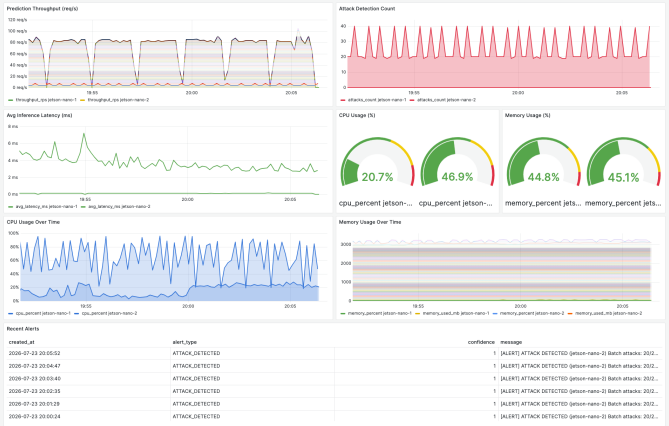
2026-09-29

A Distributed Real-Time Intrusion Detection System on a Jetson Orin Nano Super Edge Cluster using Kafka and PySpark

Backup: which model, and why

Backup: which model, and why				
Configuration	F1	Features	Size (MB)	Edge rationale
Baseline + XGBoost	0.9971	77	2.32	Reference accuracy
SHAP Top-30 + XGBoost	0.9970	30	0.003 [†]	Explainable; de- ployed feature set
RF Top-30 + XGBoost	0.9922	30	2.52	Embedded ranking
PCA k=40 + XGBoost	0.9939	40	3.82	Unsupervised reduc- tion
We deploy SHAP Top-30 with Random Forest (offline F1 0.9955): a native Spark ML estimator, so the exported PipelineModel loads on ARM64 without the native libraries Spark's XGBoost integration needs. The input vector has 29 features — destination_port is in the SHAP ranking but excluded as label-leaking.				
[†] Distributed saving under-reports size; true size ≈2.5 MB.				

Backup: full monitoring dashboard



Per-node throughput, detected attacks, inference latency, CPU/RAM for both boards,
PostgreSQL-backed alerts.

2026-09-29

A Distributed Real-Time Intrusion Detection System on a Jetson Orin Nano Super Edge Cluster using Kafka and PySpark

Backup: full monitoring dashboard

Backup: full monitoring dashboard



Per-node throughput, detected attacks, inference latency, CPU/RAM for both boards,
PostgreSQL-backed alerts.

Backup: measurement protocol

- Host-side orchestration starts pipeline roles on the Jetsons over SSH, drives the load, collects per-node logs.
- ≥ 3 repetitions per configuration; warmup traffic *excluded* from the measured window (no JVM/JIT inflation of early batches).
- Metric queries aligned to the exact load window, not a trailing range.
- Latency percentiles per node over *raw per-flow* samples; cluster p95 is conservatively the *worst node's* p95.
- End-to-end send-to-verdict latency from producer timestamps under NTP-synchronized clocks (excludes DB-write cost).
- Energy: tegrastats, 30 s idle baseline then load window; two-node modes sum both boards.

2026-09-29

A Distributed Real-Time Intrusion Detection System on a Jetson Orin Nano Super Edge Cluster using Kafka and PySpark

Backup: measurement protocol

- Host-side orchestration starts pipeline roles on the Jetsons over SSH, drives the load, collects per-node logs.
- ≥ 3 repetitions per configuration; warmup traffic excluded from the measured window (no JVM/JIT inflation of early batches).
- Metric queries aligned to the exact load window, not a trailing range.
- Latency percentiles per node over *raw per-flow* samples; cluster p95 is conservatively the *worst node's* p95.
- End-to-end send-to-verdict latency from producer timestamps under NTP-synchronized clocks (excludes DB-write cost).
- Energy: tegrastats, 30 s idle baseline then load window; two-node modes sum both boards.